AEGIS SIEM

Developer & Attribution Guide

Code architecture, implementation notes, dependencies, MITRE references, source provenance and responsible reuse.

PROJECT DOCUMENTATION • VERSION 1.0 • 31 OCTOBER 2026

Purpose

This document describes the current Aegis SIEM starter application as implemented in the delivered source files. It is intended to help an analyst operate the application, interpret its outputs, and understand its limitations.

Contents

- 1. Provenance and what is original in Aegis
- 2. Application architecture
- 3. Project file inventory
- 4. Main code modules and key functions
- 5. Detection and risk implementation
- 6. MITRE ATT&CK® content: scope and attribution
- 7. Third-party dependencies and official documentation
- 8. Testing and verification performed
- 9. Licensing, reuse and release checklist
- 10. Suggested technical roadmap

1. Provenance and what is original in Aegis

Aegis was assembled specifically for this project through an iterative ChatGPT-assisted development process. The application source (app.py and mitre.py), UI structure, field normalization, detection heuristics, risk-score aggregation, report composition and the documentation were authored for this Aegis project. The project was not created by cloning or copying a complete third-party SIEM application or a public GitHub repository.

The application uses third-party Python packages through normal imports and package installation. Their APIs and behavior are informed by their official documentation. That is distinct from copying their implementation source code into Aegis. MITRE ATT&CK technique identifiers, names, tactic labels and official reference links are drawn from the official MITRE ATT&CK knowledge base; Aegis provides its own local rule-to-technique association logic and caveats.

> **Transparent attribution statement**
>
> The Aegis-specific application code in this delivery was generated and iteratively refined for this project with AI assistance. Third-party libraries and MITRE ATT&CK content remain subject to their respective terms. No formal open-source license is included for Aegis itself.

What was not imported as a source

- No complete SIEM engine, detection pack or Streamlit dashboard template was copied wholesale.
- No MITRE ATT&CK STIX bundle, TAXII feed, Navigator layer or Sigma rule pack is bundled. The application contains a small hand-maintained mapping catalogue for its local detections.
- No remote service, API key or telemetry upload is required by the delivered analysis workflow.

2. Application architecture

The architecture is intentionally compact. app.py hosts the Streamlit interface, ingestion/normalization helpers, detection logic, stateful workspace pages and PDF report generation. mitre.py isolates the ATT&CK technique catalogue and the rule-to-technique mapping logic.

Stage	Implementation	Primary output
Input	Streamlit file uploader + pandas.read_excel	Raw DataFrame from selected worksheet.
Schema matching	normalize_name() + suggest_mapping() + user-selected mapping	Dictionary mapping canonical Aegis fields to Excel columns.

Stage	Implementation	Primary output
Normalization	normalize_logs() + normalize_severity() + classify_event()	Normalized event DataFrame with category and event_text.
Detection	analyze_logs() and nested add_finding()	Findings list, enriched events, risk score and risk level.
ATT&CK enrichment	mitre.map_finding()	Candidate technique records and explicit caveat / no-map rationale.
UI	Streamlit session state, page selector, Plotly charts, tables and download buttons	Command Center, Findings, Event Explorer, Reporting Center, Data & Rules.
PDF	make_pdf_report() using ReportLab Platypus	English or Spanish PDF byte stream for download.

Data path

```
Excel workbook / worksheet
      |
      v
pandas.read_excel -> raw DataFrame
      |
      v
suggest_mapping -> analyst confirms canonical fields
      |
      v
normalize_logs -> normalized event DataFrame
      |
      v
analyze_logs -> findings + enriched events + risk score
      |
      +--> mitre.map_finding -> candidate ATT&CK techniques / caveats
      |
      +--> Streamlit SOC workspaces + CSV exports
      |
      +--> make_pdf_report -> technical PDF
```

3. Project file inventory

File / folder	Role	Implementation notes
app.py	Main application.	Streamlit setup, palettes/fonts, CSS, translations, canonical schema, aliases, mapping helpers, normalization, detection, risk scoring, PDF report generation and workspace navigation.
mitre.py	ATT&CK alignment module.	Static technique metadata plus map_finding(); returns candidate mappings and explicit limitations. No runtime web request.
requirements.txt	Python package requirements.	Streamlit, pandas, openpyxl, xlrd, Plotly and ReportLab ranges/pins used by the Windows setup.
.streamlit/config.toml	Default Streamlit theme/server configuration.	Sets base theme and browser-facing page defaults; app-specific styling is layered in app.py.

File / folder	Role	Implementation notes
setup_windows.bat	First-time Windows setup.	Selects 64-bit Python 3.10, creates .venv, upgrades packaging tools, installs binary wheels and verifies key imports.
run_windows.bat	Application launcher.	Runs Streamlit using the project .venv Python executable.
sample_logs.xlsx	Synthetic demonstration dataset.	Contains simulated events designed to exercise several detection patterns; not real incident evidence.
README.md	Quick project setup and run guide.	Short operational reference; the full analyst guide is provided separately.
Aegis_SIEM_User_Guide.docx / .pdf	Analyst documentation.	Detailed workflow, rules, score interpretation, ATT&CK use and operating constraints.
Aegis_SIEM_Developer_and_Attribution_Guide.docx / .pdf	Technical/source documentation.	Architecture, functions, dependencies, provenance, citations and reuse notes.

4. Main code modules and key functions

app.py - visual system and localization

Function / object	Purpose
PALETTES	Defines Graphite & Jade, Midnight Blue and Light Analyst colors as theme tokens.
FONT_STACKS / FONT_LABELS	Maps selectable local font families to CSS font stacks and English/Spanish selector labels.
build_ui_css(palette, font_stack)	Builds the CSS used for navigation, surfaces, cards, sidebar controls, tables and readable contrast.
polish_chart(fig, palette, font_stack, ...)	Applies a consistent theme, font, plot background, grid, title margin, axis styling and layout to Plotly charts.
TEXT, tr(), ui_text()	Stores and returns interface strings in English and Spanish.
localized_severity(), localized_category(), finding_text()	Localizes severity/category labels and Aegis-generated finding narratives; raw source log text remains unchanged.

app.py - ingestion, normalization and detection

Function	Purpose / input-output
normalize_name(value)	Simplifies a source column label to lowercase alphanumeric words for alias matching.
suggest_mapping(columns)	Compares workbook columns against ALIASES and returns a best-effort canonical-field mapping.
_series_or_default(df, col, default)	Reads a mapped source column or returns a default-filled Series when no column was mapped.
normalize_logs(raw, mapping)	Builds the canonical event DataFrame, parses timestamps as UTC, trims text fields, converts bytes_out, creates event_text, normalizes severity and classifies events.
normalize_severity(value)	Maps common string/number scales to Critical, High, Medium, Low or Unknown. Vendor scales are not universally standardized.
classify_event(row)	Uses event IDs and message keywords to assign broad categories such as authentication failure, success, privilege change, process execution or network activity.
_event_evidence(df, indices, limit=5)	Builds up to five concise evidence strings from timestamp, source IP, user, event ID and event name/message.

Function	Purpose / input-output
analyze_logs(logs, thresholds...)	Runs the local detection rules, builds finding records, tags matched rows, assigns detection severity and returns findings + enriched log rows + score + level.
nested add_finding(...)	Deduplicates row indexes, collects entities/times/evidence, calculates a score weight, generates a finding ID, and calls map_finding() to add ATT&CK metadata.

app.py - reporting and workspace

Function / state	Purpose
make_pdf_report(findings, logs, score, level, source_name, language, metadata)	Builds the multi-page PDF through ReportLab; adds report ID, time, summary metrics, data coverage, findings/evidence, ATT&CK register, action plan, scoring and caveats.
st.session_state["analysis"]	Stores the current workbook analysis during the active Streamlit session so the workspace pages can reuse it.
st.session_state["active_page"]	Remembers which of the five workspace pages is visible.
Command Center branch	Displays aggregate score, key metrics, risk distribution, priority queue and selected analytics charts.
Findings branch	Lists findings and shows detailed explanation, evidence, ATT&CK candidate mappings and CSV export.
Event Explorer branch	Searches and filters normalized event rows, including detected-only filter and CSV download.
Reporting Center branch	Shows report metadata and summary, then creates the PDF for download.
Data & Rules branch	Loads the workbook, lets the user choose the sheet and adjust column mapping, exposes thresholds and starts the analysis.

mitre.py - ATT&CK reference logic

ATTACK_REFERENCE_AS_OF records the date the technique metadata was reviewed. TECHNIQUES stores IDs, names, tactic labels, official URLs, rationale and Spanish rationale. map_finding(rule, title, event_texts, event_ids) maps a specific local finding pattern to zero or more candidate techniques and returns assessment wording plus an explicit mapping caveat. technique_catalog() exposes a shallow copy for tests or documentation.

This module is deliberately not a general-purpose ATT&CK mapper. It does not parse arbitrary logs to infer every possible technique, fetch live ATT&CK STIX data, track ATT&CK version changes automatically, or calculate technique confidence. Its output is a manually curated candidate association between Aegis rule patterns and ATT&CK technique references.

5. Detection and risk implementation

The detection implementation is deterministic and rule-based. It inspects normalized strings, event IDs, timestamps, source IPs, usernames, destination IPs, outbound-byte values and vendor severity. A detection should be considered an investigation lead that requires context and corroboration.

Rule	Core implementation detail
AUTH-BURST	Groups failed authentications using the available source_ip and username dimensions. With timestamps, searches a rolling 10-minute window for the configured count; without usable timestamps, uses a count-only fallback.

Rule	Core implementation detail
AUTH-SUCCESS-AFTER-FAIL	For a successful event with usable timestamp, selects prior failed events within 30 minutes and filters on source and/or username when present; at least three related failures trigger the finding.
EXEC-SUSPICIOUS	Regular-expression patterns for encoded PowerShell, download cradles, certutil/BITSAdmin transfer, selected LOLBins, credential-dumping keywords and shadow-copy/backup deletion.
DEFENSE-EVASION-LOG-CLEAR	Pattern match for Event IDs 1102/104 coupled with clear/log wording and common log-clear phrases.
IDENTITY-PRIVILEGE	Flags IDs 4672, 4728, 4732, 4756 and selected privilege/group-membership phrases.
NETWORK-FANOUT	Groups rows with destination IP by source IP and counts unique destination IPs across the whole export; configured threshold defaults to 20.
DATA-LARGE-EGRESS	Compares each numeric bytes_out value to exfil_mb * 1024 * 1024; default exfil_mb is 100.
VENDOR-HIGH-SEVERITY	When enabled, carries forward uncovered rows with normalized vendor severity High or Critical; it avoids rows already tagged by another local rule.

Risk-score calculation

```
severity_points = {Critical: 30, High: 22, Medium: 13, Low: 6, other: 4}
volume_factor = min(1.7, 1 + log10(max(1, finding_count)) * 0.18)
score_weight = severity_points * volume_factor
risk_score = min(100, round(sum(score_weight)))
```

Risk bands are 0 Informational, 1-24 Low, 25-49 Medium, 50-74 High and 75-100 Critical. Because the model is additive and capped, overlapping rules can drive the score to 100. ATT&CK mapping is informational and does not affect risk calculation.

6. MITRE ATT&CK® content: scope and attribution

Aegis uses official MITRE ATT&CK® terminology for the technique names, IDs, tactic labels and links listed in mitre.py. The mapping rationale and rule matching logic are Aegis-specific. The application does not claim that the MITRE organization created, validated or endorsed the Aegis detections.

The current catalogue includes Password Guessing (T1110.001), PowerShell (T1059.001), Command Obfuscation (T1027.010), Ingress Tool Transfer (T1105), BITS Jobs (T1197), selected Windows utility proxy execution techniques, Visual Basic (T1059.005), LSASS Memory (T1003.001), Inhibit System Recovery (T1490), Clear Windows Event Logs (T1685.005), Additional Local or Domain Groups (T1098.007), and Network Service Discovery (T1046). See the analyst guide for each association and its evidence limitations.

Required MITRE copyright designation

© 2026 The MITRE Corporation. This work is reproduced and distributed with the permission of The MITRE Corporation.

MITRE requires its first written reference to use “MITRE ATT&CK®” and its branding guidance prohibits implying affiliation, sponsorship or endorsement. Aegis therefore uses a clear independent-project disclaimer. The MITRE designation above is included in the generated PDF report and should remain in distributed copies that reproduce ATT&CK material.

The log-clearing mapping uses the official current technique T1685.005 - Clear Windows Event Logs (tactic: Defense Impairment). This is intentionally not labeled with the older technique ID used in earlier versions of the framework. Recheck the official technique page and update ATTACK_REFERENCE_AS_OF when revising the catalogue.

7. Third-party dependencies and official documentation

The packages below are third-party dependencies installed by requirements.txt. Aegis imports their public interfaces; their own source code is installed by pip into the virtual environment, not embedded into the Aegis source files.

Dependency	Pinned / bounded requirement	Role in Aegis
Python	3.10.x, 64-bit for supplied Windows scripts.	Runtime and standard library.
Streamlit	>=1.39, <2.0	Web UI, sidebar controls, file upload, state, page reruns, tables and download buttons.
pandas	==2.3.3	Excel worksheet ingestion and tabular normalization, grouping, filtering and analysis.
openpyxl	>=3.1, <4.0	Excel .xlsx reading engine used with pandas.
xlrd	>=2.0, <3.0	Legacy .xls workbook reading engine used with pandas.
Plotly	>=5.20, <7.0	Interactive charts and chart layout objects.
ReportLab	>=4.0, <5.0	Programmatic PDF report generation with paragraphs, tables and page templates.

Official documentation and project references

1. Streamlit documentation - UI framework and components. https://docs.streamlit.io/

2. Streamlit Session State - Session state and rerun behavior. https://docs.streamlit.io/develop/concepts/architecture/session-state

3. pandas read_excel - Excel file ingestion. https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html

4. openpyxl documentation - XLSX workbook reading engine. https://openpyxl.readthedocs.io/en/stable/

5. xlrd documentation - Legacy XLS workbook reading engine. https://xlrd.readthedocs.io/en/latest/

6. Plotly Python documentation - Interactive charting. https://plotly.com/python/

7. Plotly Graph Objects - Figure and layout interface. https://plotly.com/python/graph-objects/

8. ReportLab documentation - PDF generation library. https://docs.reportlab.com/

9. Python documentation - Language runtime and standard library. https://docs.python.org/3.10/

10. MITRE ATT&CK Legal & Branding - Trademark and attribution requirements. https://attack.mitre.org/resources/legal-and-branding/

11. MITRE ATT&CK Terms of Use - Permission and required copyright designation. https://attack.mitre.org/resources/terms-of-use/

Package license and project license notes

Each dependency is distributed under its own license. Before redistributing a packaged environment or a modified application, review the license and notice files supplied with the exact installed package versions and include any required notices. The requirements file records dependency versions/ranges but is not a substitute for license review.

No Aegis LICENSE file is included in this delivery. Therefore, the project should not be described as being under MIT, Apache-2.0, GPL or another open-source license unless the maintainer deliberately chooses and adds one. A code delivery does not by itself establish permission for unrestricted public reuse. Choose a license and add an attribution/NOTICE file before publishing a public repository.

8. Testing and verification performed

The packaged code was syntax-compiled. A smoke-test harness loaded the included synthetic workbook, exercised the mapping/normalization/detection functions and generated English and Spanish PDF reports. The test dataset contains 38 rows and currently generates six findings; five receive candidate ATT&CK mappings and the large-egress-only signal remains unmapped by design.

Test area	Result / coverage
Python syntax	app.py and mitre.py compile successfully.
Synthetic Excel workflow	38 event rows read and normalized; six findings generated.
Candidate ATT&CK mapping	Authentication, encoded PowerShell, log-clear and network fan-out cases map; the large-egress-only case remains unmapped.
PDF generation	English and Spanish reports generated; each rendered to four pages and visually checked for clipping, overlap and broken table layout.
Browser UI	Not launched in this container because Streamlit is not installed in the authoring environment. The app was designed for the working Python 3.10 installation previously used by the project maintainer; verify the browser interface after patching locally.

Test data is not production validation

The sample workbook is synthetic. Passing a smoke test demonstrates that the code path executes on the included cases; it does not establish detection quality, false-positive rate, false-negative rate, performance at scale or suitability for production monitoring.

9. Licensing, reuse and release checklist

- Retain the MITRE copyright designation and independent-project disclaimer in materials that reproduce ATT&CK content.
- Keep the “candidate alignment” language and mapping caveats; do not present a technique as confirmed from a keyword match alone.
- Review third-party package licenses/notices for the exact versions installed and include notices if redistribution requires them.
- Choose an Aegis project license before publishing the source publicly; do not imply a license that is not included.
- Replace synthetic test data with a clearly marked anonymized/synthetic-only example; never publish real customer logs or personal data.
- Document the Python and dependency versions used for release; build/test the package on a clean Windows machine.
- Add unit tests for every detection threshold, time-zone edge case, missing field, malformed workbook and ATT&CK mapping decision.
- Use a versioned rule catalogue and record the date/ATT&CK revision used for each release.

10. Suggested technical roadmap

Priority	Enhancement	Why it matters
1	Rule unit tests and test fixtures for positive, negative and boundary cases.	Makes detections reproducible and reduces regressions when rules change.
2	Vendor-specific import profiles for Sentinel, Splunk, QRadar, Elastic, Wazuh and common Windows exports.	Improves field mappings and event interpretation.
3	Rule metadata: source requirements, detection confidence, ATT&CK data source, references and tuning guidance.	Makes the detection catalogue auditable and maintainable.
4	Analyst case workflow: status, owner, comments, evidence references and timeline.	Moves the app beyond one-off workbook triage.
5	Persistent database, authenticated access, RBAC, audit logging and secure deployment controls.	Required before multi-user or production hosting.
6	ATT&CK catalogue update process and optional Navigator export.	Helps keep technique metadata current and makes coverage easier to communicate.
7	Optional enrichments such as DNS/proxy/EDR joins and approved threat intelligence.	Adds context; should be explicit, source-attributed and privacy-reviewed.

Closing note

Aegis is a portfolio-grade prototype when presented honestly: an explainable Excel-based security triage workbench with visual analytics, candidate ATT&CK associations and structured reporting. The strongest demonstration is not the dashboard alone, but the ability to explain a finding, show exactly which source rows triggered it, describe the rule and its limits, map behavior conservatively, and recommend validation steps.

References and official resources

The following links are official project documentation or framework references. Aegis uses these resources for API behavior, format support, and technique nomenclature; it does not embed or redistribute their full source projects.

[1] MITRE ATT&CK® - Legal & Branding. Trademark and no-endorsement guidance.
https://attack.mitre.org/resources/legal-and-branding/

[2] MITRE ATT&CK® - Terms of Use. License terms and required copyright designation.
https://attack.mitre.org/resources/terms-of-use/

[3] MITRE ATT&CK® - Clear Windows Event Logs. Official current reference for T1685.005.
https://attack.mitre.org/techniques/T1685/005/

[4] Streamlit - Session State. Explains per-session state and rerun behavior.
https://docs.streamlit.io/develop/concepts/architecture/session-state

[5] pandas - read_excel. Excel ingestion API and engine selection.
https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html

[6] Plotly - Graph Objects. Interactive figure model. https://plotly.com/python/graph-objects/

[7] ReportLab - Platypus. Flow-based report layout and table building.
https://docs.reportlab.com/reportlab/userguide/ch5_platypus/